

# 技術主管深入追問・問答預演

## 比基礎題更深一層的 9 題，針對你的技術主張做壓力測試

這些是技術主管聽完你的說法後，最可能往下鑽的問題。答題策略都一樣：先承認取捨、再給做法、最後用數據或安全收尾。每題附「傳達訊號」，提醒你這樣答是在讓對方相信什麼。

### Q1 你說 RAG 是主力——當檢索回來的內容互相矛盾，或根本找不到對的資料時，Agent 怎麼處理？

**答** 我會先承認「檢索不一定可靠」是 RAG 的本質風險，不假裝它完美。具體做幾件事：檢索後加一層相關性過濾，低於門檻就不採用；要求模型回答時標註依據來源，讓矛盾被攤開而不是被模型抹平；真的找不到夠好的依據時，寧可讓 Agent 回「目前資料不足」並轉人工，也不要硬掰。對企業來說，「老實說不知道」比「自信講錯」安全得多。

**傳達訊號：**展現你懂 RAG 的失敗模式，而且把「安全」放在「好看」前面——這是企業用 AI 最在意的。

### Q2 知識圖譜聽起來很美，但建構與維護成本很高、schema 又常變動，你怎麼避免它變成負債？

**答** 這是好問題，我不會假裝圖譜沒有代價。它最貴的就是建構與 schema 維護——schema 一變，整套關係要跟著改，很容易變負債。所以我的原則是：不要一開始就上圖譜。先用向量庫滿足多數模糊檢索，只有當「關係查詢」真的成為痛點（例如多跳的供應鏈或製程追溯）才導入，而且範圍要小、schema 要克制。讓圖譜只解決它最擅長的問題，而不是什麼都往裡塞。

**傳達訊號：**誠實承認自己技術選擇的代價，反而比硬吹更可信；也顯示你不會為了用新技術而用。

### Q3 你用 LLM-as-judge 來評估——但 judge 本身也會出錯，你怎麼相信評估結果？

**答** 對，judge 不是真理，所以我不會只靠它。做法是：judge 只當「便宜的大規模初篩」，再用一小批人工標註的 golden set 去校準它的準確度——等於先確認這把尺準不準，再拿它來量。高風險場景的關鍵指標，我仍會保留人工抽檢。重點是把評估當成「逐步逼近」，而不是一個能完全自動化的銀彈。

**傳達訊號：**顯示你對評估方法有批判性思考，不會天真地相信任何單一工具。

#### Q4 多步 Agent 一步錯就步步錯，錯誤會累積，你怎麼控制？

**答** 這是 Agent 最現實的問題。對策有三：第一，縮短鏈路——能三步完成就不要五步，步數越少累積誤差越小。第二，每一步加驗證——關鍵步驟的輸出先用程式或規則檢查，不過關就重試或停下。第三，加 checkpoint 與反思，讓 Agent 階段性檢查「目前方向對嗎」。本質就是：早點發現錯，別讓它滾雪球。

**傳達訊號：***「能少一步就少一步」展現你不是為複雜而複雜，懂得用簡單換穩定。*

#### Q5 製造業要地端，但地端能跑的 7B-14B 模型能力不如雲端大模型，你怎麼補這個落差？

**答** 我研究過地端硬體（像 RTX 5090、DGX Spark）跑 7B-14B 的取舍，所以很清楚落差在哪。補的方式不是硬要小模型變聰明，而是用系統設計補模型能力：把難任務拆成小任務，讓小模型也做得動；用 RAG 把知識餵進去，而不是要它記住；對特定領域用 LoRA / QLoRA 微調，讓它在窄領域追上大模型；少數真的需要高難度推理的步驟，再走雲端。地端優先處理敏感與高頻，雲端處理少數難題，兼顧安全、成本與能力。

**傳達訊號：***把你的硬體研究跟微調經驗直接變成解法，這題正好打在你的強項上。*

#### Q6 很多人一窩蜂上 Agent。你怎麼判斷一個任務該用 Agent，還是用固定流程（workflow）就好？

**答** 我對這點很有意見——Agent 的自主性是有代價的：不穩、難測、又貴。我的判準很簡單：流程固定、步驟明確的，用 workflow 甚至傳統程式就好，不要用 Agent；只有當「下一步要做什麼，得看上一步的結果臨場判斷」時，才值得用 Agent 的自主決策。能用簡單方法解決就別動用大砲，這對成本跟穩定性都好。

**傳達訊號：***展現工程判斷力與成本意識——資深工程師的標誌是「克制」，不是什麼都上最潮的。*

#### Q7 Prompt 改一個字輸出就變，這麼脆弱的東西要上線，你怎麼做版本管理與回歸？

**答** 正因為脆弱，所以不能手改完就上線。我會把 prompt 當程式碼管理：進版控、寫清楚每次改了什麼、為什麼改；每次改動都跑回歸測試（用前面說的 eval set），比對分數有沒有退化；上線採漸進式，先小流量觀察再全面放開。等於把「調 prompt」從手工藝，變成有測試保護的工程流程。

**傳達訊號：**把 LLM 開發拉回扎實的軟體工程紀律，正好呼應你「規格化」的主軸。

### Q8 內部 Agent 接了一堆工具，prompt injection 跟資料外洩的風險怎麼防？

**答** 工具越多風險越高，所以要分層防護。幾道防線：權限最小化——Agent 只拿到完成任務必要的工具與資料，會寫入、刪除的高風險操作要人確認；輸入防護——對外來內容（文件、網頁）保持警覺，避免它夾帶指令操控 Agent；所有工具呼叫與輸出做稽核紀錄，可追溯；敏感資料優先地端處理，不外流到外部 API。安全不是單一機制，是一層一層疊起來的。

**傳達訊號：**顯示你把安全當設計的一部分，而不是事後補丁——製造業對這點特別敏感。

### Q9 新版模型一直出，你怎麼決定要不要換？換了會不會出事？

**答** 新模型不代表要馬上換——換模型像換引擎，可能更快，也可能水土不服。我的做法：先用既有的 eval set 把新舊模型跑同一批考題，比對品質、延遲、成本；再用小流量做 A / B，確認真的更好才全面換；過程中保留快速回滾的能力。重點是用數據決定，而不是追新。

**傳達訊號：**再次強調「用數據決定、保留回滾」，給技術主管一種「這個人上線不會出包」的安心感。